



MODUL PEMROGRAMAN BERBASIS OBJEK 1

Pertemuan Ke-2

PHYTON BASE

Disusun Oleh :

Aziz Azindani, S.Pd., S,Kom., M.Kom

Modul Perkuliahan: Pertemuan 2

Enkapsulasi dan Abstraksi

Tujuan Pembelajaran:

1. Memahami dan menerapkan konsep enkapsulasi dalam OOP.
 2. Memahami dan menerapkan konsep abstraksi dalam OOP.
 3. Menggunakan akses modifier untuk melindungi data.
 4. Membuat dan menggunakan kelas abstrak dengan modul `abc`.
-

I. Pengantar

Pada pertemuan ini, kita akan mempelajari dua konsep penting dalam OOP: **enkapsulasi** dan **abstraksi**. Keduanya berperan dalam menyederhanakan kompleksitas sistem dan melindungi data dalam aplikasi.

1. Enkapsulasi

Enkapsulasi adalah proses menyembunyikan detail implementasi dari pengguna dan hanya menyediakan antarmuka publik untuk berinteraksi dengan objek. Ini dilakukan dengan menggunakan akses modifier.

Akses Modifier:

- **Private:** Hanya dapat diakses dari dalam kelas. Digunakan untuk menyembunyikan atribut atau metode.
- **Protected:** Dapat diakses dalam kelas dan subkelas. Digunakan untuk memberikan akses yang lebih terbatas.
- **Public:** Dapat diakses dari mana saja. Biasanya digunakan untuk metode yang harus diakses dari luar.

Metode Getter dan Setter

Getter dan Setter adalah metode yang digunakan untuk mengakses dan mengubah nilai atribut private.

Contoh:

```
class Mahasiswa:
    def __init__(self, nama, npm):
        self.__nama = nama
        self.__npm = npm

    def get_nama(self):
        return self.__nama
```

```
def set_nama(self, nama):
    self.__nama = nama

# Praktik
mhs1 = Mahasiswa("Alice", "123456")
print(mhs1.get_nama())
mhs1.set_nama("Bob")
print(mhs1.get_nama())
```

2. Abstraksi

Abstraksi menyembunyikan detail yang tidak relevan dari pengguna dan hanya menampilkan fitur penting dari objek. Abstraksi sering diimplementasikan melalui kelas abstrak dan metode abstrak.

Membuat Kelas Abstrak

Python menyediakan modul `abc` untuk membuat kelas abstrak dan metode abstrak.

Contoh:

```
from abc import ABC, abstractmethod

class Kendaraan(ABC):
    @abstractmethod
    def bergerak(self):
        pass

class Mobil(Kendaraan):
    def bergerak(self):
        print("Mobil bergerak di jalan")

# Praktik
mobil = Mobil()
mobil.bergerak()
```

II. Enkapsulasi: Teori dan Praktik Terintegrasi

A. Definisi dan Penerapan Enkapsulasi

Enkapsulasi adalah penyembunyian data dengan menyimpan atribut dan metode dalam kelas. Ini mencegah akses tidak langsung ke data dengan menggunakan akses modifier.

1. Akses Modifier

- **Private:** Digunakan untuk melindungi data dari akses luar. Ditandai dengan dua garis bawah (`__`).
 - **Contoh:**

```
class AkunBank:
    def __init__(self, saldo):
        self.__saldo = saldo
```

- **Protected:** Memberikan akses terbatas kepada kelas turunan. Ditandai dengan satu garis bawah (_).
 - **Contoh:**

```
class Pegawai:
    def __init__(self, nama, gaji):
        self._gaji = gaji
```

- **Public:** Akses penuh dari mana saja. Tidak ada tanda khusus.
 - **Contoh:**

```
class Produk:
    def __init__(self, nama, harga):
        self.nama = nama
        self.harga = harga
```

2. Penggunaan Getter dan Setter

Getter dan Setter digunakan untuk memberikan cara aman dalam mengakses dan mengubah data private.

- **Contoh Implementasi Getter dan Setter:**

```
class AkunBank:
    def __init__(self, saldo_awal):
        self.__saldo = saldo_awal

    def get_saldo(self):
        return self.__saldo

    def set_saldo(self, jumlah):
        if jumlah >= 0:
            self.__saldo = jumlah
        else:
            print("Saldo tidak boleh negatif")

# Praktik
akun = AkunBank(1000)
print(akun.get_saldo())
akun.set_saldo(1500)
print(akun.get_saldo())
akun.set_saldo(-500) # Harusnya memberikan pesan error
```

3. Contoh Real Enkapsulasi dalam Kehidupan Nyata

Pikirkan aplikasi perbankan di mana data nasabah (seperti saldo) dilindungi dari perubahan langsung. Enkapsulasi memastikan bahwa pengguna tidak dapat langsung memodifikasi saldo tanpa melalui metode yang sesuai (seperti deposit atau withdraw).

III. Abstraksi: Teori dan Praktik Terintegrasi

A. Definisi dan Penerapan Abstraksi

Abstraksi adalah proses menyederhanakan kompleksitas dengan mengabaikan detail yang tidak relevan dan hanya menampilkan fitur penting dari objek.

1. Kelas Abstrak dan Metode Abstrak

Kelas abstrak adalah kelas yang tidak dapat diinstansiasi dan biasanya digunakan sebagai blueprint untuk kelas turunan. Metode abstrak adalah metode yang didefinisikan dalam kelas abstrak tanpa implementasi dan harus diimplementasikan oleh kelas turunan.

- **Contoh Membuat Kelas Abstrak:**

```
from abc import ABC, abstractmethod

class Kendaraan(ABC):
    @abstractmethod
    def bergerak(self):
        pass

class Mobil(Kendaraan):
    def bergerak(self):
        print("Mobil bergerak di jalan")

# Praktik
mobil = Mobil()
mobil.bergerak()
```

2. Penerapan Abstraksi dalam Kehidupan Nyata

Bayangkan konsep `Kendaraan` yang bisa berupa `Mobil`, `Motor`, atau `Pesawat`. Kita tidak perlu tahu bagaimana setiap jenis kendaraan bergerak secara detail, kita hanya peduli bahwa setiap kendaraan bisa bergerak. Kelas abstrak `Kendaraan` mendefinisikan bahwa setiap kendaraan harus memiliki kemampuan `bergerak`.

3. Langkah-Langkah Membuat dan Menggunakan Kelas Abstrak

- **Definisikan Kelas Abstrak dengan Metode Abstrak:**

```
from abc import ABC, abstractmethod

class Hewan(ABC):
    @abstractmethod
    def suara(self):
        pass
```

- **Turunkan Kelas Abstrak dan Implementasikan Metode Abstrak:**

```
class Kucing(Hewan):
    def suara(self):
        print("Kucing mengeong")
```

```

class Anjing(Hewan):
    def suara(self):
        print("Anjing menggonggong")

# Praktik
kucing = Kucing()
kucing.suara()
anjing = Anjing()
anjing.suara()

```

IV. Latihan dan Praktik Mandiri

A. Praktik Mandiri: Enkapsulasi dengan Kelas `AkunBank`

Tugas:

1. Buat kelas `AkunBank` dengan atribut private `saldo`.
2. Tambahkan metode untuk set dan get saldo dengan validasi agar saldo tidak negatif.

Contoh Solusi:

```

class AkunBank:
    def __init__(self, saldo_awal):
        self.__saldo = saldo_awal

    def get_saldo(self):
        return self.__saldo

    def set_saldo(self, jumlah):
        if jumlah >= 0:
            self.__saldo = jumlah
        else:
            print("Saldo tidak boleh negatif")

# Praktik
akun = AkunBank(1000)
print(akun.get_saldo())
akun.set_saldo(2000)
print(akun.get_saldo())
akun.set_saldo(-500) # Harusnya memberikan pesan error

```

B. Praktik Mandiri: Abstraksi dengan Kelas `Hewan`

Tugas:

1. Buat kelas abstrak `Hewan` dengan metode abstrak `suara`.
2. Buat kelas `Kucing` dan `Anjing` yang mengimplementasikan `Hewan`.

Contoh Solusi:

```

from abc import ABC, abstractmethod

class Hewan(ABC):

```

```
@abstractmethod
def suara(self):
    pass

class Kucing(Hewan):
    def suara(self):
        print("Kucing mengeong")

class Anjing(Hewan):
    def suara(self):
        print("Anjing menggonggong")

# Praktik
kucing = Kucing()
kucing.suara()
anjing = Anjing()
anjing.suara()
```

V. Rangkuman

1. **Enkapsulasi** melindungi data dengan membatasi akses langsung ke atribut dan menyediakan metode untuk memanipulasi data secara aman.
 2. **Abstraksi** menyederhanakan kompleksitas sistem dengan menyediakan antarmuka yang konsisten dan menyembunyikan detail implementasi.
 3. **Getter dan Setter** adalah cara umum untuk mengimplementasikan enkapsulasi dalam kelas.
 4. **Kelas Abstrak** digunakan sebagai blueprint untuk kelas turunan yang mengharuskan implementasi metode tertentu.
-

VI. Pertanyaan dan Diskusi

1. Bagaimana enkapsulasi dapat membantu dalam mengelola perubahan dalam aplikasi?
 2. Apa perbedaan utama antara enkapsulasi dan abstraksi?
 3. Diskusikan contoh nyata di mana abstraksi dapat menyederhanakan desain sistem.
-

VII. Lampiran Kode

Kode Enkapsulasi:

```
class Mahasiswa:
    def __init__(self, nama, npm):
        self.__nama = nama
        self.__npm = npm

    def get_nama(self):
        return self.__nama

    def set_nama(self, nama):
        self.__nama = nama
```

```
mhs1 = Mahasiswa("Alice", "123456")
print(mhs1.get_nama())
mhs1.set_nama("Bob")
print(mhs1.get_nama())
```

Kode Abstraksi:

```
from abc import ABC, abstractmethod

class Kendaraan(ABC):
    @abstractmethod
    def bergerak(self):
        pass

class Mobil(Kendaraan):
    def bergerak(self):
        print("Mobil bergerak di jalan")

mobil = Mobil()
mobil.bergerak()
```

Kode Latihan:

```
class AkunBank:
    def __init__(self, saldo_awal):
        self.__saldo = saldo_awal

    def get_saldo(self):
        return self.__saldo

    def set_saldo(self, jumlah):
        if jumlah >= 0:
            self.__saldo = jumlah
        else:
            print("Saldo tidak boleh negatif")

from abc import ABC, abstractmethod

class Hewan(ABC):
    @abstractmethod
    def suara(self):
        pass

class Kucing(Hewan):
    def suara(self):
        print("Kucing mengeong")

class Anjing(Hewan):
    def suara(self):
        print("Anjing menggonggong")

kucing = Kucing()
kucing.suara()
anjing = Anjing()
anjing.suara()
```